Section 5

Micro – CC421

Assembly

Structure of An Assembly program

```
.model small
.stack 128
.data

.code
main proc far

    MOV AX,@DATA
    MOV DS,AX



    mov ax, 4c00h ; exit to operating system.
    int 21h

main endp
end main ; set entry point and stop the assembler.
```

Size of Program

Size of Stack

Data

Code

Models

SMALL is one of the most widely used memory models for Assembly language programs and is sufficient for the programs in this book. The small model uses a maximum of 64K bytes of memory for code and another 64K bytes for data. The other models are defined as follows:

```
.MODEL MEDIUM      ;the data must fit into 64K bytes
                   ;but the code can exceed 64K bytes of memory
.MODEL COMPACT     ;the data can exceed 64K bytes
                   ;but the code cannot exceed 64K bytes
.MODEL LARGE       ;both data and code can exceed 64K
                   ;but no single set of data should exceed 64K
.MODEL HUGE        ;both code and data can exceed 64K
                   ;data items (such as arrays) can exceed 64K
.MODEL TINY        ;used with COM files in which data and code
                   ;must fit into 64K bytes
```

How function works in C programming?

```
#include <stdio.h>

void functionName()
{
    ... .. ...
    ... .. ...
}

int main()
{
    ... .. ...
    ... .. ...

    functionName();

    ... .. ...
    ... .. ...
}
```

Procedure

1. In some applications it is common practice to save all registers at the beginning of a subroutine. Assume that SP= 1288H before a subroutine CALL. Show the contents of the stack pointer and the exact memory contents of the stack after PUSHF for the following:

```
1132:0450 CALL PROC1
1132:0453 INC BX
PROC1 PROC
        PUSH AX
        PUSH BX
        PUSH CX
        PUSH DX
        PUSH  SI
        PUSH DI
        PUSHF
        ....
PROC1 ENDP
```

Procedure

1. In some applications it is common practice to save all registers at the beginning of a subroutine. Assume that SP= 1288H before a subroutine CALL. Show the contents of the stack pointer and the exact memory contents of the stack after PUSHF for the following:

```
1132:0450 CALL PROC1
1132:0453 INC BX
PROC1 PROC
        PUSH AX
        PUSH BX
        PUSH CX
        PUSH DX
        PUSH  SI
        PUSH DI
        PUSHF
        ....
PROC1 ENDP
```

Procedure

1. In some applications it is common practice to save all registers at the beginning of a subroutine. Assume that SP= 1288H before a subroutine CALL. Show the contents of the stack pointer and the exact memory contents of the stack after PUSHF for the following:

```
1132:0450 CALL PROC1
1132:0453 INC BX
PROC1 PROC
        PUSH AX
        PUSH BX
        PUSH CX
        PUSH DX
        PUSH  SI
        PUSH DI
        PUSHF
        ....
PROC1 ENDP
```

PUSH 0453 Automatically

Procedure

1. In some applications it is common practice to save all registers at the beginning of a subroutine. Assume that SP= 1288H before a subroutine CALL. Show the contents of the stack pointer and the exact memory contents of the stack after PUSHF for the following:

```
1132:0450 CALL PROC1
1132:0453 INC BX
PROC1 PROC        PUSH 0453 Automatically   SP=1286
        PUSH AX   SP=1284
        PUSH BX   SP=1282
        PUSH CX   SP=1280
        PUSH DX   SP=127E
        PUSH  SI  SP=127C
        PUSH DI   SP=127A
        PUSHF     SP=1278
        ....
PROC1 ENDP
```

To restore the original information inside the CPU at the end of a CALL to a subroutine, the sequence of POP instructions must follow a certain order. Write the sequence of POP instructions that will restore the information in Problem 1. At each point, show the contents of the SP.

To restore the original information inside the CPU at the end of a CALL to a subroutine, the sequence of POP instructions must follow a certain order. Write the sequence of POP instructions that will restore the information in Problem 1. At each point, show the contents of the SP.

Popf
Pop di
Pop si
Pop dx
Pop cx
Pop bx
Pop ax

To restore the original information inside the CPU at the end of a CALL to a subroutine, the sequence of POP instructions must follow a certain order. Write the sequence of POP instructions that will restore the information in Problem 1. At each point, show the contents of the SP.

Popf SP=127A
Pop di SP=127C
Pop si SP=127E
Pop dx SP=1280
Pop cx SP=1282
Pop bx SP=1284
Pop ax SP=1286

To restore the original information inside the CPU at the end of a CALL to a subroutine, the sequence of POP instructions must follow a certain order. Write the sequence of POP instructions that will restore the information in Problem 1. At each point, show the contents of the SP.

Popf SP=127A
Pop di SP=127C
Pop si SP=127E
Pop dx SP=1280
Pop cx SP=1282
Pop bx SP=1284
Pop ax SP=1286

PROC1 ENDP POP IP (0453) Automatically SP=1288

When a CALL is executed, how does the CPU know where to return?

In a FAR CALL, _____ and _____ are saved on the stack, where as in a NEAR CALL, _____ is saved on the stack.

When a CALL is executed, how does the CPU know where to return?

In a FAR CALL, ____CS____ and ____IP____ are saved on the stack, where as in a NEAR CALL, ____IP____ is saved on the stack.

Compare the number of bytes of stack taken due to NEAR and FAR CALLs.

Compare the number of bytes of stack taken due to NEAR and FAR CALLs.

NEAR: 2 Bytes (Push only IP)
FAR: 4 Bytes (Push CS and IP)

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP     Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8      C8: MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Verify the address calculations of:

(a) JNC ERROR1 (b) JNO ERROR1 (c) JMPC8

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

IP	Code	
E06C	733F	JNC ERROR1
...	...	
E072	7139	JNO ERROR1
...	...	
E08C	8ED8	C8: MOV DS,AX
...	...	
E0A7	EBE3	JMP C8
...	...	
E0AD	F4	ERROR1: HLT

Verify the address calculations of:

(a) JNC ERROR1 (b) JNO ERROR1 (c) JMPC8

Code

73 3F JNC ERROR1

Op code

Displacement
Of the jump

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP    Code
E06C  733F        JNC  ERROR1
...   ...
E072  7139        JNO  ERROR1
...   ...
E08C  8ED8     C8: MOV  DS,AX
...   ...
E0A7  EBE3        JMP  C8
...   ...
E0AD  F4          ERROR1: HLT
```

Let's check the displacement of the first jump:

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP    Code
E06C  733F        JNC  ERROR1
...   ...
E072  7139        JNO  ERROR1
...   ...
E08C  8ED8     C8: MOV  DS,AX
...   ...
E0A7  EBE3        JMP  C8
...   ...
E0AD  F4          ERROR1: HLT
```

Let's check the displacement of the first jump:

+ve jump (coming instruction)
Displacement = Target – IP

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP    Code
E06C  733F        JNC  ERROR1
...   ...
E072  7139        JNO  ERROR1
...   ...
E08C  8ED8     C8: MOV  DS,AX
...   ...
E0A7  EBE3        JMP  C8
...   ...
E0AD  F4          ERROR1: HLT
```

Let's check the displacement of the first jump:

+ve jump (coming instruction)
Displacement = Target – IP

Address of the tag (If the jump happened)

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP    Code
E06C  733F        JNC  ERROR1
...   ...
E072  7139        JNO  ERROR1
...   ...
E08C  8ED8        C8: MOV  DS,AX
...   ...
E0A7  EBE3        JMP  C8
...   ...
E0AD  F4          ERROR1: HLT
```

Let's check the displacement of the first jump:

+ve jump (coming instruction)
Displacement = Target – IP

Next instructio n to be fetched

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP     Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8      C8: MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the first jump:

+ve jump (coming instruction)
Displacement = Target – IP

Displacement = E0AD – E06E
= 3F

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP     Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8      C8: MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the first jump:

+ve jump (coming instruction)
Displacement = Target – IP

Displacement = E0AD – E06E
= 3F

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP     Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8      C8: MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the second jump:

+ve jump (coming instruction)
Displacement = Target – IP

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP     Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8      C8: MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the second jump:

+ve jump (coming instruction)
Displacement = Target – IP

Displacement = E0AD – E074
= 39

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP    Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8      C8: MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the second jump:

+ve jump (coming instruction)
Displacement = Target – IP

Displacement = E0AD – E074
= 39

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP     Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8     C8:  MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the 3rd jump:

-ve jump (previous instruction)
Displacement = IP – Target

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP     Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8     C8:  MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the second jump:

-ve jump (previous instruction)
Displacement = IP – Target
= E0A9 – E08C
= 1D

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP    Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8     C8:  MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the second jump:

-ve jump (previous instruction)
Displacement = IP – Target
= E0A9 – E08C
= 1D

Must indicate it's a –ve jump

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP    Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8     C8:  MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the second jump:

-ve jump (previous instruction)
Displacement = IP – Target
= E0A9 – E08C = 1D

Must indicate it's a –ve jump
2's comp of (1D)
1D = 0001 1101
1110 0010 (1's)
1110 0011 (2's) => E3

7. The following is a section of BIOS of the IBM PC, which is described in detail in Chapter 3. All the jumps below are short jumps, meaning that the labels are in the range –128 to +127.

```
IP    Code
E06C 733F          JNC  ERROR1
...  ...
E072 7139          JNO  ERROR1
...  ...
E08C 8ED8     C8:  MOV  DS,AX
...  ...
E0A7 EBE3          JMP  C8
...  ...
E0AD F4            ERROR1: HLT
```

Let's check the displacement of the second jump:

-ve jump (previous instruction)
Displacement = IP – Target
= E0A9 – E08C = 1D

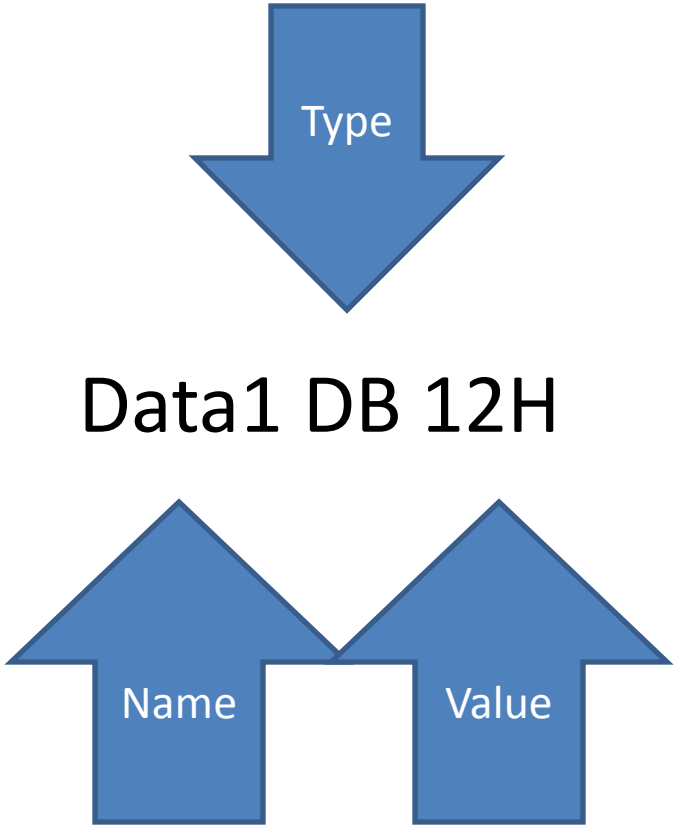
Must indicate it's a –ve jump
2's comp of (1D)
1D = 0001 1101
1110 0010 (1's)
1110 0011 (2's) => E3

+ve jump **Displacement = Target - IP**	**-ve jump** **Displacement = 2's comp (IP – Target)**

Data

ORG (origin)

ORG is used to indicate the beginning of the offset address. The number that comes after ORG can be either in hex or in decimal. If the number is not followed by H, it is decimal and the assembler will convert it to hex. Although the ORG directive is used extensively in this book in the data segment to separate fields of data to make it more readable for the student, it can also be used for the offset of the code segment (IP).


Type
Data1 DB 12H
Name
Value

Types (1)

DB (define byte)

The DB directive is one of the most widely used data directives in the assembler. It allows allocation of memory in byte-sized chunks. This is indeed the smallest allocation unit permitted. DB can be used to define numbers in decimal, binary, hex, and ASCII. For decimal, the D after the decimal number is optional, but using B (binary) and H (hexadecimal) for the others is required. Regardless of which one is used, the assembler will convert them into hex. To indicate ASCII, simply place the string in single quotation marks ('like this'). The assembler will assign the ASCII code for the numbers or characters automatically. DB is the only directive that can be used to define ASCII strings larger than two characters; therefore, it should be used for all ASCII data definitions. Following are some DB examples:

DB examples

```
DATA1   DB   25            ;DECIMAL
DATA2   DB   10001001B     ;BINARY
DATA3   DB   12H           ;HEX
        ORG  0010H
DATA4   DB   '2591'        ;ASCII NUMBERS
        ORG  0018H
DATA5   DB   ?             ;SET ASIDE A BYTE
        ORG  0020H
DATA6   DB   'My name is Joe' ;ASCII CHARACTERS
```

Types (2)

DW (define word)

DW is used to allocate memory 2 bytes (one word) at a time. DW is used widely in the 8088/8086 and 80286 microprocessors since the registers are 16 bits wide. The following are some examples of DW:

DW example

```
        ORG   70H
DATA11  DW    954                    ;DECIMAL
DATA12  DW    100101010100B          ;BINARY
DATA13  DW    253FH                  ;HEX
        ORG   78H
DATA14  DW    9,2,7,0CH,00100000B,5,'HI'   ;MISC. DATA
```

Types (3)

DD (define doubleword)

The DD directive is used to allocate memory locations that are 4 bytes (two words) in size. Again, the data can be in decimal, binary, or hex. In any case the data is converted to hex and placed in memory locations according to the rule of low byte to low address and high byte to high address.

DD examples

```
        ORG  00A0H
DATA16  DD   1023                          ;DECIMAL
DATA17  DD   10001001011001011100B         ;BINARY
DATA18  DD   5C2A57F2H                     ;HEX
DATA19  DD   23H,34789H,65533
```

Types (4)

DQ (define quadword)

DQ is used to allocate memory 8 bytes (four words) in size. This can be used to represent any variable up to 64 bits wide:

DQ examples

```
        ORG  00C0H
DATA20  DQ   4523C2H     ;HEX
DATA21  DQ   'HI'        ;ASCII CHARACTERS
DATA22  DQ   ?           ;NOTHING
```

Types (4)

DT (define ten bytes)

DT is used for memory allocation of packed BCD numbers. The application of DT will be seen in the multibyte addition of BCD numbers in Chapter 3. For now, observe how they are located in memory. Notice that the "H" after the data is not needed. This allocates 10 bytes, but a maximum of 18 digits can be entered.

DT Examples

```
        ORG  00E0H
DATA23  DT   867943569829   ;BCD
DATA24  DT   ?              ;NOTHING
```

Equate

EQU (equate)

This is used to define a constant without occupying a memory location. EQU does not set aside storage for a data item but associates a constant value with a data label so that when the label appears in the program, its constant value will be substituted for the label. EQU can also be used outside the data segment, even in the middle of a code segment. Using EQU for the counter constant in the immediate addressing mode:

```
COUNT  EQU 25
```

When executing the instructions "MOV CX,COUNT", the register CX will be loaded with the value 25.

Duplicate

DUP (duplicate)

DUP is used to duplicate a given number of characters. This can avoid a lot of typing. For example, contrast the following two methods of filling six memory locations with FFH:

```
        ORG   0030H
DATA7   DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;FILL 6 BYTES WITH FF
        ORG   38H
DATA8   DB    6 DUP(0FFH)     ;FILL 6 BYTES WITH FF
```

Question 1

Find the precise offset location of each ASCII character or data in the following:

```
ORG 20H
DATA1 DB '1-800-555-1234'
ORG 40H
DATA2 DB 'Name: John Jones'
ORG 60H
DATA3 DB '5956342'
ORG 70H
DATA4 DW 2560H,1000000000110B
DATA5 DW 49
ORG 80H
DATA6 DD 25697F6EH
DATA7 DQ 9E7BA21C99F2H
ORG 90H
DATA8 DT 439997924999828
DATA9 DB 6 DUP (0EEH)
```

Answer

```
ORG 20H
DATA1 DB '1-800-555-1234'
```

Off	20H	21H	22H	23H	24H	…			
Data	‘1’	‘-’	‘8’	‘0’	‘0’	…			

```
ORG 40H
DATA2 DB 'Name: John Jones'
```

40h	41h	42h	43h	44h	45h	46h	.....		
‘N’	‘a’	‘m’	‘e’	‘:’	‘ ‘	‘J’	.....		

```
ORG 60H
DATA3 DB '5956342'
```

60h	61h	62h	63h	64h	65h	66h			
‘5’	‘9’	‘5’	‘6’	‘3’	‘4’	‘2’			

```
ORG 70H
DATA4 DW 2560H,1000000000110B
DATA5 DW 49
```

70h	71h	72h	73h	74h	75h				
60h	25h	00000110b	00010000b	49	0				

```
ORG 80H
DATA6 DD 25697F6EH
DATA7 DQ 9E7BA21C99F2H
```

80h	81h	82h	83h	84h	85h	86h	87h	88h
6E	7F	69	25	F2	99	1C	A2	7B

89h	8Ah	8Bh						
9E	00H	00H						

```
ORG 90H
DATA8 DT 439997924999828
DATA9 DB 6 DUP (0EEH)
```

90H	91H	92H	93H	94H	95H	96H	97H	98H
28	98	99	24	79	99	39	04	00

99H	9AH	9BH	9CH	9DH	9EH	9FH		
00	0EEH	0EEH	0EEH	0EEH	0EEH	0EEH		